

Universidade Federal do Ceará
Campus Quixadá
QXD0068 – Reuso de Software
Prof. Francisco Victor da Silva Pinheiro

Trabalho 1
Refatoração com Padrões de Projeto em Sistema Legado - Sistema Ombi

Alunos: Vitor Manoel Alves da Silva

Guilherme Damasceno Nobre

Quixadá-CE
20256

Sumário

1. Introdução
2. Caracterização e Diagnóstico do Sistema Legado
3. Padrões de Projeto Selecionados
4. Refatorações Realizadas
5. Análise dos Resultados
6. Conclusão
7. Referências

1. Introdução

Reuso de software é a prática de construir sistemas a partir de componentes que já existem, em vez de escrever tudo de novo a cada necessidade. Isso exige um código organizado de forma a separar o que é estável do que varia. Em sistemas que crescem durante anos, essa separação costuma se perder. As decisões locais fazem sentido quando são tomadas, mas vão se somando até produzir classes grandes demais, trechos repetidos em vários pontos e dependências diretas entre partes que deveriam ser independentes.

Refatoração é a técnica usada para corrigir esse quadro. Segundo Fowler (2018), refatorar é alterar a estrutura interna de um software sem mudar seu comportamento observável. Os padrões catalogados por Gamma et al. (1995) servem de alvo para essa transformação, já que cada um isola um tipo específico de variação.

O sistema escolhido para este trabalho foi o Ombi, uma aplicação web de código aberto escrita em C# sobre .NET 8. O Ombi funciona como uma camada de intermediação para servidores de mídia pessoais: usuários pedem filmes, séries e álbuns através de uma interface web; um administrador aprova ou recusa; os pedidos aprovados são encaminhados automaticamente para gerenciadores de download como Sonarr, Radarr e SickRage; e, quando o conteúdo passa a existir no servidor de mídia do usuário, que pode ser Plex, Emby ou Jellyfin, o sistema marca o pedido como atendido e avisa quem pediu.

A escolha foi feita a partir do CodeTriage, um catálogo de projetos de código aberto que buscam contribuidores. O porte pesou bastante: com 23 projetos na solution, o Ombi tem problemas de projeto reais sem ser grande demais para uma dupla entender dentro do prazo. A portabilidade também contou, já que o projeto compila em Windows, Linux e macOS e usa SQLite por padrão, o que permitiu que os integrantes trabalhassem em sistemas operacionais diferentes. O que decidiu, porém, foram os sete projetos de teste automatizado que já vêm no repositório. Sem eles não haveria como verificar se o comportamento sobreviveu às mudanças.

O objetivo do trabalho foi identificar problemas concretos de projeto no código do Ombi e corrigi-los com padrões GoF, preservando o comportamento do sistema e medindo o efeito das mudanças sobre acoplamento, coesão, duplicação e extensibilidade. Foram aplicados seis padrões, das três categorias do catálogo original.

O repositório utilizado está disponível em https://github.com/GuilhermeDNobre/Ombi. A versão original do sistema está preservada na tag v-original, que aponta para o commit c564409, e a versão refatorada está na branch refactored. A comparação completa entre as duas pode ser obtida com o comando git diff v-original..refactored.

2. Caracterização e Diagnóstico do Sistema Legado

2.1 Caracterização do Sistema

O Ombi automatiza um ciclo que sem ele é manual. Em um servidor de mídia compartilhado, quem quer assistir a algo fora do catálogo pede ao administrador, que procura o conteúdo e o adiciona. As funcionalidades principais são a busca de títulos em bases externas como TheMovieDb, o registro e o acompanhamento de pedidos, um fluxo de aprovação com limites de cota por usuário, o encaminhamento automático dos pedidos aprovados aos gerenciadores de download, a sincronização periódica do catálogo já disponível nos servidores de mídia e o envio de notificações por quatorze canais diferentes, entre eles e-mail, Discord, Telegram, Slack e webhooks genéricos.
O back-end é escrito em C# sobre .NET 8, usa Entity Framework Core para persistência, Quartz para o agendamento das tarefas periódicas e SQLite como banco padrão. O front-end é uma aplicação Angular servida pelo próprio ASP.NET Core. A solução está organizada em 23 projetos, dos quais sete são de teste automatizado, escritos com NUnit e Moq.
A tabela a seguir descreve os módulos relevantes para este trabalho.

Tabela 1 – Módulos do sistema analisados

Módulo	Responsabilidade
Ombi.Core/Engine	Regras de negócio dos pedidos: validação, duplicidade, aprovação, cotas, filtros e ordenação
Ombi.Core/Senders	Encaminhamento dos pedidos aprovados aos gerenciadores de download
Ombi.Notifications	Quatorze agentes de notificação derivados de uma classe base abstrata
Ombi.Schedule/Jobs	Tarefas periódicas, com um conjunto de jobs por servidor de mídia
Ombi.Api.External	Clientes HTTP das APIs de terceiros
Ombi.DependencyInjection	Registro de todos os serviços no contêiner de injeção de dependência

Uma característica do sistema orientou o diagnóstico: o Ombi já usa injeção de dependência por construtor em quase todas as suas classes, e a camada de notificações já aplica uma forma de Template Method. Os problemas encontrados são de outra natureza. São abstrações mal dimensionadas, duplicação entre integrações equivalentes e classes que foram acumulando responsabilidades ao longo das versões.

2.2 Problemas Identificados

Dependência rígida de implementações concretas no envio aos gerenciadores de download. A classe TvSender, com 633 linhas, verificava em sequência cada gerenciador suportado dentro de um único método, e cada verificação trazia seu próprio tratamento de erro. O trecho abaixo mostra o padrão:

```
var sonarr = await SonarrSettings.GetSettingsAsync();
if (sonarr.Enabled)
{
    var result = await SendToSonarr(model, sonarr);
    if (result != null)
    {
        return new SenderResult { Sent = true, Success = true };
    }
}

var sr = await SickRageSettings.GetSettingsAsync();
if (sr.Enabled)
{
    var result = await SendToSickRage(model, sr);
    if (result)
    {
        return new SenderResult { Sent = true, Success = true };
    }
    senderResult = new SenderResult { Message = "Could not send to SickRage!" };
}
```

O construtor da classe recebia ISonarrV3Api e ISickRageApi diretamente, junto com os serviços de configuração de cada um. A classe conhecia em tempo de compilação todos os destinos possíveis, de modo que acrescentar um gerenciador exigia modificá-la, violando o princípio aberto/fechado. A mesma estrutura aparecia em MovieSender, com 294 linhas, que lidava com Radarr, Radarr 4K, DogNzb e CouchPotato.

Acoplamento entre as regras de negócio e o canal de notificação. Os motores de pedidos acionavam o serviço de notificação diretamente ao aprovar, recusar ou marcar um item como disponível. O disparo era feito pela classe NotificationHelper, que chamava o agendador do Quartz explicitamente dentro de cada método:

```
await OmbiQuartz.TriggerJob(nameof(INotificationService), "Notifications",
    new Dictionary<string, object>
    {
        { JobDataKeys.NotificationOptions, notificationModel }
    });
```

Esse bloco se repetia em todos os métodos do helper. A regra de negócio ficava amarrada ao mecanismo de entrega da notificação, e não apenas ao fato de ela existir. Testar a regra exigia simular o agendador. Qualquer outro interessado nesses eventos, como auditoria ou coleta de métricas, obrigaria a alterar o helper.

Duplicação entre integrações equivalentes. O Ombi sincroniza o catálogo de três servidores de mídia, e cada um tinha o seu próprio job executando o mesmo algoritmo: abrir a conexão, percorrer as bibliotecas, processar séries, processar filmes e remover o conteúdo que saiu do catálogo. Emby e Jellyfin, em particular, tinham a mesma sequência de métodos com os tipos trocados.

Tabela 2 – Jobs de sincronização na versão original

Job	Linhas	Métodos principais
PlexContentSync.cs	907	StartTheCache, ProcessServer, MovieLoop, ProcessTvShow
EmbyContentSync.cs	371	ProcessMovies, RemoveStaleContent
JellyfinContentSync.cs	331	StartServerCache, ProcessTv, ProcessMovies

Não existia nenhuma abstração de servidor de mídia no sistema. Cada job falava diretamente com a API do seu servidor e com os modelos de dados próprios dela, de modo que uma correção no algoritmo de sincronização precisava ser replicada três vezes.

Classe com responsabilidades excessivas. A classe MovieRequestEngine, com 887 linhas, concentrava a validação da entrada, a checagem de duplicidade, as regras de aprovação, o controle de cota, a persistência, o disparo de notificações, o envio ao gerenciador de download e ainda a montagem dos modelos de exibição. Além disso, a lógica de filtro e ordenação estava escrita como blocos switch sobre enumerações, com quatro ocorrências somente nessa classe, nas linhas 223, 237, 317 e 436:

```
switch (orderFilter.AvailabilityFilter)
{
    case FilterType.None:
        break;
    case FilterType.Available:
        allRequests = allRequests.Where(x => x.Available);
        break;
    ...
}
```

A mesma estrutura se repetia em MusicRequestEngine e TvRequestEngine, esta última com 1.095 linhas. Com tantos motivos de mudança reunidos, a classe virou o ponto único de alteração do módulo. Quase toda evolução do sistema passava por ela.

3. Padrões de Projeto Selecionados

Foram selecionados seis padrões do catálogo GoF. Cada um veio depois do diagnóstico, escolhido para um problema específico da seção anterior. A tabela abaixo resume a correspondência.

Tabela 3 – Padrões aplicados

Padrão	Categoria	Problema atacado
Strategy	Comportamental	Condicionais encadeadas escolhendo o gerenciador de download
Factory Method	Criacional	Criação e seleção das estratégias espalhada pelo sender
Template Method	Comportamental	Algoritmo de sincronização replicado por servidor
Adapter	Estrutural	APIs de servidores de mídia sem interface comum
Observer	Comportamental	Regras de negócio acopladas ao canal de notificação
Facade	Estrutural	Motor de pedidos com responsabilidades excessivas

Strategy. Padrão comportamental que encapsula algoritmos intercambiáveis em classes separadas. Cada gerenciador de download é exatamente isso: uma forma alternativa de executar a mesma operação, escolhida em tempo de execução conforme a configuração do usuário. As classes envolvidas são as interfaces ITvDvrSender e IMovieDvrSender e as implementações SonarrDvrSender, SickRageDvrSender, RadarrDvrSender, Radarr4KDvrSender, CouchPotatoDvrSender e DogNzbDvrSender. Com isso, acrescentar um gerenciador deixa de exigir alteração no código existente.

Factory Method. Padrão criacional que delega a criação de objetos a um ponto especializado. Complementa o Strategy: decidir quais estratégias estão habilitadas e em que ordem tentá-las não é tarefa do chamador. As classes envolvidas são TvDvrSenderFactory e MovieDvrSenderFactory. A lógica de seleção passa a existir em um lugar só.

Template Method. Padrão comportamental que mantém o esqueleto do algoritmo na classe base e deixa passos específicos para as subclasses. O ciclo de sincronização é idêntico entre os servidores; o que muda são os detalhes de cada passo. A classe envolvida é MediaServerContentSync, que concentra o algoritmo antes espalhado pelos três jobs.
Adapter. Padrão estrutural que converte a interface de uma classe naquela que o cliente espera. As APIs do Emby e do Jellyfin têm contratos e modelos próprios, incompatíveis entre si, e o template precisa de um contrato único. As classes envolvidas são a interface IMediaServerCatalog e os adaptadores EmbyCatalogAdapter e JellyfinCatalogAdapter. Suportar um servidor novo passa a custar uma classe adaptadora.

Observer. Padrão comportamental em que os interessados são avisados quando o estado do objeto observado muda. Notificar é reação a um evento de domínio, não parte da regra de aprovação. As classes envolvidas são a interface IRequestEventObserver, a classe NotificationHelper, que passa a publicar, e NotificationService, que passa a observar. A regra de negócio fica livre do canal de saída.

Facade. Padrão estrutural que oferece uma interface única para um conjunto de interfaces de um subsistema. As responsabilidades do motor de pedidos precisavam ser separadas sem quebrar os controladores que já o consomem. As classes envolvidas são MovieRequestEngine, na posição de fachada, e os serviços MovieRequestFactory, MovieRequestStatusService, MovieRequestDispatcher, MovieRequestEnricher e MovieRequestQueryBuilder. A coesão sobe sem que a mudança se propague para a camada de API.

4. Refatorações Realizadas

Cada refatoração foi desenvolvida em uma branch própria e integrada por pull request, o que permite acompanhar a evolução de cada uma separadamente no histórico. A comparação entre a versão original e a refatorada envolve 62 arquivos, com 4.009 linhas acrescentadas e 1.935 removidas.

4.1 Padrão 1 – Strategy e Factory Method

Situação original. TvSender e MovieSender concentravam o conhecimento de todos os gerenciadores suportados, o código de comunicação com cada API, o tratamento de erro de cada integração e a lógica de fila de falhas.

Problema identificado. Dependência rígida de implementações concretas, com violação do princípio aberto/fechado, conforme detalhado na seção 2.2.
Solução implementada. Cada bloco de envio foi extraído para uma classe que implementa ITvDvrSender ou IMovieDvrSender, com os métodos IsEnabledAsync, Send e a propriedade Priority. As fábricas consultam as configurações e devolvem apenas as estratégias habilitadas, já na ordem de prioridade. O sender passou a iterar sobre essa lista. Como Radarr e Radarr 4K compartilham quase todo o comportamento, foi criada ainda a classe RadarrDvrSenderBase, reaproveitada pelas duas estratégias.
Depois:

```
public async Task<SenderResult> Send(ChildRequests model)
{
    var senders = await _factory.GetEnabledSendersAsync();
    if (!senders.Any())
    {
        return new SenderResult { Success = true };
    }

    SenderResult lastFailure = null;
    foreach (var sender in senders)
    {
        var result = await sender.Send(model);
        if (result.Success)
        {
            return result;
        }
        lastFailure = result;
    }

    await AddToRequestFailureQueue(model, lastFailure.Message);
    return lastFailure;
}
```

O construtor da classe, que antes recebia as APIs concretas, passou a receber apenas a fábrica, o repositório da fila e o helper de notificação. As estratégias são registradas no contêiner de injeção de dependência, e a fábrica as recebe como uma coleção:

```
services.AddTransient<ITvDvrSender, SonarrDvrSender>();
services.AddTransient<ITvDvrSender, SickRageDvrSender>();
services.AddTransient<ITvDvrSenderFactory, TvDvrSenderFactory>();
```

Resultado. TvSender caiu de 633 para 83 linhas e MovieSender de 294 para 83. Acrescentar um gerenciador novo virou escrever uma classe e registrar uma linha.

4.2 Padrão 2 – Template Method e Adapter

Situação original. Três jobs independentes, descritos na Tabela 2, implementando o mesmo ciclo de sincronização para servidores diferentes.

Problema identificado. Duplicação estrutural entre integrações equivalentes, sem nenhuma abstração comum de servidor de mídia.

Solução implementada. Foi criada a interface genérica IMediaServerCatalog, que expressa o que o domínio precisa de um servidor de mídia, independentemente de qual seja:

```
public interface IMediaServerCatalog<TContent> where TContent : MediaServerContent
{
    string ServerName { get; }
    int PageSize { get; }

    Task<IReadOnlyList<MediaServerInstance>> OpenAsync();
    Task<MediaServerPage<MediaServerSeries>> GetShows(...);
    Task<MediaServerPage<MediaServerMovie>> GetMovies(...);
    Task<IReadOnlyList<MediaServerMovie>> GetCollection(...);

    Task NotifySyncStarting(bool recentlyAdded);
    Task NotifySyncFinished();
}
```

Os adaptadores EmbyCatalogAdapter e JellyfinCatalogAdapter traduzem as respostas de cada API para os modelos comuns MediaServerMovie, MediaServerSeries, MediaServerLibrary e MediaServerPage. A classe abstrata MediaServerContentSync define a sequência do algoritmo e delega os passos variáveis ao catálogo, com pontos de extensão opcionais como OnSyncCompleted e StopOnEmptyPage para as diferenças entre servidores.

O job do Jellyfin, que tinha 331 linhas, ficou assim por inteiro:

```
public class JellyfinContentSync : MediaServerContentSync<JellyfinContent>,
IJellyfinContentSync
{
    public JellyfinContentSync(
        IMediaServerCatalog<JellyfinContent> catalog,
        IJellyfinContentRepository repo,
        ILogger<JellyfinContentSync> logger) : base(catalog, repo, logger)
    {
    }

    protected override async Task OnSyncCompleted()
    {
        await OmbiQuartz.TriggerJob(nameof(IJellyfinEpisodeSync), "Jellyfin");
```

```
    }
}
```

Resultado. JellyfinContentSync caiu de 331 para 27 linhas e EmbyContentSync de 371 para 139. A diferença entre os dois vem de particularidades do Emby que permaneceram como sobrescritas. O algoritmo de sincronização passou a existir uma única vez.

4.3 Padrão 3 – Observer

Situação original. O NotificationHelper repetia, em cada um de seus métodos, a chamada explícita ao agendador do Quartz para disparar o serviço de notificação.

Problema identificado. Acoplamento entre a regra de negócio e o mecanismo de entrega da notificação.

Solução implementada. Foi criada a interface IRequestEventObserver, com um único método Handle. O NotificationHelper passou a receber a coleção de observadores pelo construtor e a publicar os eventos para todos eles, sem saber quem são nem o que farão:

```
public NotificationHelper(IEnumerable<IRequestEventObserver> observers)
{
    _observers = observers;
}

public async Task NewRequest(FullBaseRequest model)
{
    var notificationModel = new NotificationOptions { ... };
    await PublishAsync(notificationModel);
}
```

O NotificationService passou a implementar IRequestEventObserver, assumindo a responsabilidade de acionar o agendador, que antes estava espalhada pelo helper. O registro no contêiner é o que liga os dois lados:

```
services.AddTransient<IRequestEventObserver, NotificationService>();
```

Resultado. A regra de negócio publica um evento em vez de acionar um canal específico. Para acrescentar outro interessado nos eventos de pedido, como auditoria ou métricas, basta registrar mais um observador, sem tocar no helper nem nos motores.

4.4 Padrão 4 – Facade

Situação original. A classe MovieRequestEngine, com 887 linhas, acumulando sete responsabilidades distintas.

Problema identificado. Baixa coesão e concentração de motivos de mudança em uma única classe.

Solução implementada. As responsabilidades foram extraídas para cinco serviços, cada um com uma interface própria, reunidos no namespace Ombi.Core.Engine.Requests:

Serviço	Responsabilidade extraída
MovieRequestFactory	Construção e persistência do pedido
MovieRequestStatusService	Aprovação, recusa e disponibilidade
MovieRequestDispatcher	Envio ao gerenciador de download
MovieRequestEnricher	Montagem dos modelos de exibição
MovieRequestQueryBuilder	Filtro e ordenação, substituindo os blocos switch

A classe original permaneceu no lugar, com a mesma interface pública IMovieRequestEngine, passando a atuar como fachada que coordena esses colaboradores. O método de criação de pedido, por exemplo, reduziu-se a delegar e tratar o retorno:

```
public async Task<RequestEngineResult> RequestMovie(MovieRequestViewModel model)
{
    var buildResult = await _factory.Build(model);
    if (buildResult.Error != null)
    {
        return buildResult.Error;
    }
    ...
}
```

Resultado. MovieRequestEngine caiu de 887 para 430 linhas, uma redução de 51%. Como a interface consumida pelos controladores não mudou, nenhuma alteração foi necessária na camada de API.

5. Análise dos Resultados

A comparação entre as duas versões olhou para o tamanho e a distribuição do código, para a estrutura de dependências e para a cobertura de testes.

Tabela 4 – Linhas de código das classes afetadas

Classe	Antes	Depois	Variação
TvSender.cs	633	83	−87%
MovieSender.cs	294	83	−72%
MovieRequestEngine.cs	887	430	−51%
EmbyContentSync.cs	371	139	−63%
JellyfinContentSync.cs	331	27	−92%

O código não desapareceu. Ele foi redistribuído em 35 classes de produção novas, cada uma com um propósito único. A tabela mede a saída da concentração para a distribuição de responsabilidades.

Redução de acoplamento. A medida mais direta está nos construtores. TvSender dependia de quatro tipos concretos ligados a integrações específicas e passou a depender de três abstrações, nenhuma delas presa a um fornecedor. O NotificationHelper deixou de conhecer o agendador do Quartz e passou a conhecer apenas a interface dos observadores. Os jobs de sincronização deixaram de falar com as APIs do Emby e do Jellyfin e passam a falar com IMediaServerCatalog.

Melhoria da coesão. As cinco classes extraídas do motor de pedidos têm, cada uma, um motivo único de mudança. Uma alteração na regra de cota afeta o serviço de status; uma mudança na ordenação afeta o construtor de consultas. Antes, as duas mexiam na mesma classe.

Redução de duplicação. O ciclo de sincronização, que existia em três cópias, passou a existir uma vez em MediaServerContentSync. Na família Radarr, o comportamento comum entre as versões padrão e 4K foi reunido em RadarrDvrSenderBase.

Extensibilidade. Alguns cenários de evolução ficaram mais baratos. Um gerenciador de download novo é uma classe que implementa a interface de estratégia, mais uma linha de registro. Um servidor de mídia novo é um adaptador e uma subclasse mínima do template. Um interessado novo nos eventos de pedido é um observador registrado. Nenhum desses casos exige editar código existente.

Reutilização de componentes. Além dos casos citados, IMediaServerCatalog e os modelos comuns de mídia podem ser aproveitados por outras partes do sistema que hoje consultam o catálogo por

conta própria, como os verificadores de disponibilidade.
Testabilidade. Na versão original, testar o envio a um gerenciador específico exigiria instanciar o sender inteiro com todas as suas dependências. Depois da refatoração, cada estratégia é testável sozinha. Foram escritos onze novos arquivos de teste, somando 35 casos, entre eles SonarrDvrSenderTests, SickRageDvrSenderTests, RadarrDvrSenderTests, DogNzbDvrSenderTests, CouchPotatoDvrSenderTests, os testes das duas fábricas, MovieSenderTests, TvSenderTests, NotificationHelperTests e NotificationServiceHandleTests. Nenhum deles seria possível antes, porque as unidades que eles testam não existiam separadamente.
Validação do comportamento. A suíte de testes do próprio projeto foi executada na tag v-original e na versão refatorada, na mesma máquina e com o mesmo SDK.

Tabela 5 – Execução da suíte de testes

Versão	Aprovados	Falhas	Ignorados
Original (v-original)	683	6	5
Refatorada	718	6	5

As seis falhas aparecem nas duas execuções e são as mesmas, todas em NotificationMessageCurlysTests. Elas comparam textos traduzidos, como “Filmes”, “Séries” e “Álbum”, com os valores em inglês devolvidos no ambiente de teste, e decorrem da cultura configurada na máquina. São anteriores à refatoração e não têm relação com as mudanças feitas. O total de testes aprovados subiu de 683 para 718, e a diferença corresponde aos 35 casos acrescentados sobre as classes extraídas. Além da suíte automatizada, as funcionalidades principais foram verificadas por testes de ponta a ponta com Playwright sobre a aplicação em execução, cobrindo o fluxo de pedido, aprovação e acompanhamento de status.

6. Conclusão

Este trabalho aplicou seis padrões de projeto GoF ao Ombi, um sistema legado real em C#, a partir de quatro problemas identificados por inspeção do código: dependência rígida de implementações concretas no envio aos gerenciadores de download, acoplamento entre regras de negócio e canal de notificação, duplicação entre os jobs de sincronização dos servidores de mídia e concentração de responsabilidades no motor de pedidos de filmes.
Os ganhos aparecem em números. As classes que concentravam decisões encolheram entre 51% e 92%, com o comportamento redistribuído em 35 classes menores. Cenários de evolução que antes exigiam editar classes existentes passaram a ser resolvidos por acréscimo. E onze arquivos de teste puderam ser escritos sobre unidades que antes não eram isoláveis.
O reuso apareceu de duas formas. Dentro do sistema, componentes como RadarrDvrSenderBase e MediaServerContentSync passaram a ser compartilhados por implementações distintas. Para fora, as interfaces criadas, em especial IMediaServerCatalog, definem contratos que integrações futuras

podem implementar sem que o código que as consome precise saber delas.

Duas limitações devem ser registradas. A primeira é de escopo. O Facade foi aplicado apenas ao motor de pedidos de filmes, e o TvRequestEngine continua com suas 1.095 linhas originais. A decisão foi deliberada: preferimos concluir uma refatoração inteira a deixar duas pela metade. A segunda é o job de sincronização do Plex, que não foi migrado para o template. Ele tem comportamentos próprios, como a varredura de conteúdo recente e a resolução de identificadores externos, que exigiriam pontos de extensão adicionais na classe base.

O aprendizado mais útil da equipe foi sobre por onde começar. A tentação inicial é escolher os padrões e sair procurando onde encaixá-los, o que produz abstração sem propósito. O caminho que funcionou foi o contrário: ler o código, descrever o incômodo concreto e só então perguntar qual padrão resolve aquilo. Em dois casos a resposta acabou sendo uma combinação de padrões. O outro aprendizado tem a ver com a rede de segurança. Foi a suíte de testes já existente que deu confiança para mexer em classes de centenas de linhas sem medo de quebrar o comportamento.

7. Referências

FOWLER, M. *Refactoring: Improving the Design of Existing Code*. 2. ed. Boston: Addison-Wesley, 2018.

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley, 1995.

MARTIN, R. C. *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River: Prentice Hall, 2008.

MARTIN, R. C. *Agile Software Development: Principles, Patterns, and Practices*. Upper Saddle River: Prentice Hall, 2002.

MICROSOFT. *Dependency injection in .NET*. Disponível em: https://learn.microsoft.com/dotnet/core/extensions/dependency-injection. Acesso em: out. 2026.

OMBI. *Ombi: Want a Movie or TV Show on Plex, Emby or Jellyfin?*. Disponível em: https://github.com/Ombi-app/Ombi. Acesso em: out. 2026.

CODETRIAGE. *Open source projects looking for contributors*. Disponível em: https://www.codetriage.com. Acesso em: set. 2026.